

Python Programming

A practical course covering Python fundamentals, control flow, data structures, functions, strings, exceptions, files, object-oriented programming, modules, virtual environments, and problem solving.

1. Python Fundamentals

Variables and Data Types

Python is dynamically typed. Common built-in types include int, float, str, bool, list, tuple, set, and dict.

Operators

Python supports arithmetic operators such as +, -, *, /, //, %, and **; comparison operators; and logical operators and, or, and not.

Input and Output

input() reads user input as text and print() displays output. Convert input explicitly with int(), float(), or other constructors when required.

2. Conditional Statements and Loops

if, elif, else

Conditional statements execute different blocks based on Boolean conditions.

for Loop

A for loop iterates over an iterable such as a list, string, tuple, dictionary, or range.

while Loop

A while loop repeats while its condition is true. break exits a loop and continue skips to the next iteration.

3. Data Structures

Lists

Lists are ordered and mutable. Common methods include append(), extend(), insert(), remove(), pop(), sort(), and reverse().

Tuples

Tuples are ordered and immutable collections.

Sets

Sets contain unique elements and support union, intersection, and difference.

Dictionaries

Dictionaries store key-value pairs. Common methods include get(), keys(), values(), items(), update(), and pop().

4. Functions

Defining Functions

Functions are defined with `def` and may accept positional, keyword, default, and variable-length arguments.

Return Values

`return` sends a value back to the caller. A function without an explicit `return` returns `None`.

Lambda

A lambda is a small anonymous function often used with `map()`, `filter()`, and `sorted()`.

5. Strings

Operations

Strings support indexing, slicing, concatenation, repetition, and membership testing.

Methods

Useful methods include `lower()`, `upper()`, `strip()`, `split()`, `join()`, `replace()`, `find()`, `startswith()`, `endswith()`, and `count()`.

6. Comprehensions

List Comprehension

List comprehensions provide a compact way to construct lists from iterables, optionally using a condition.

Dictionary and Set Comprehensions

Similar compact syntax can construct dictionaries and sets.

7. Exception Handling

try and except

Use `try` and `except` to handle runtime exceptions.

else and finally

`else` runs when no exception occurs; `finally` runs regardless of whether an exception occurred.

raise

`raise` explicitly generates an exception when a program detects an invalid state or input.

8. File Handling

Reading and Writing

`open()` opens files. Common modes are `r` for reading, `w` for writing, and `a` for appending.

with Statement

The `with` statement is preferred for files because it automatically closes the file.

9. Object-Oriented Programming

Classes and Objects

A class defines structure and behavior. Objects are instances of classes. `__init__` initializes instance attributes.

Inheritance

Inheritance allows a class to reuse and extend behavior from another class.

Encapsulation and Polymorphism

Encapsulation organizes data and behavior. Polymorphism allows different objects to respond through a common interface.

10. Modules and Virtual Environments

Modules

Python code can be organized into modules and packages. `import` and `from ... import ...` access functionality from them.

Virtual Environments

A virtual environment isolates project dependencies. Create it, activate it, and install packages with `pip`.

11. Problem Solving

Algorithms

Common Python problem-solving techniques include loops, conditionals, functions, recursion, sorting, searching, and data structures.

Complexity

Big-O describes algorithm growth. A single pass through n elements is commonly $O(n)$, while two nested loops are commonly $O(n^2)$.

12. Practice Questions

Palindrome

Write a function that returns whether a string is a palindrome.

Largest Value

Find the largest value in a list without using `max()`.

Frequency Counter

Count the frequency of each word in a sentence using a dictionary.

Student Class

Create a Student class with name and marks and a method that calculates the average.

File Statistics

Read a text file and count its lines, words, and characters.